

SoSe 26 Type Theory & Logic & Functional Programming Notes

Igor Dimitrov

2026-04-21

Table of contents

Preface	3
I Logic	4
1 Lecture 1 - Propositional Logic: Syntax	5
1.1 Overview	5
1.2 Inductive Approach	5
1.3 Constructor Approach	5
1.4 Set Theoretic Approach	7
1.5 Summary	8
2 Lecture 2 - Height and Subformulas	9
2.1 Overview	9
2.2 Recursive Set Construction	9
2.3 Summary	12
References	13
3 Resources	14
3.1 SML	14
3.2 Haskell	14
3.3 OCAML	14
3.4 Rocq / Coq	15

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

Part I

Logic

1 Lecture 1 - Propositional Logic: Syntax

1.1 Overview

Brief summary of what this lecture does (2–4 lines).

Goal: Define the expression we can write, these are called **well-formed-formulas**, or **wffs**.

There are various ways of defining what counts as a wff. One of them is the inductive approach.

1.2 Inductive Approach

First we start with **atomic formulas**:

$$P_0, P_1, P_2, \dots$$
$$(P_i)_{i \in \mathbb{N}}$$

are atomic propositions.

We give some rules to inductively define new formulas from old ones.

- 1) if φ, ψ are **wff** then $\varphi \Rightarrow \psi$ is a **wff**
- 2) if φ, ψ are **wff** then $\varphi \wedge \psi$ is a **wff**
- 3) if φ, ψ are **wff** then $\varphi \vee \psi$ is a **wff**
- 4) if φ is a **wff** then $\neg\varphi$ is a **wff**

1.3 Constructor Approach

It is useful to express these rules with **constructors**, which define wffs as values of some functions (with codomain WFF)

In this approach atomic formulas are values of functions:

Definition 1.1 (Atomic formulas). Atomic formulas are values of functions like:

$$P : \mathbb{N} \rightarrow \text{WFF}, \quad : i \mapsto P_i$$

or

$$"." : \text{String} \rightarrow \text{WFF}, \quad : x \mapsto "x"$$

Example 1.1 (String Constructor). with the latter we can construct arbitrary propositions like:

“today is sunny”

We also have constructors for the logical connectives:

Definition 1.2 (Constructors for Logical Connectives).

$$\begin{aligned} (\neg) : \text{WFF} &\rightarrow \text{WFF}, & : \varphi &\mapsto \neg\varphi \\ (\wedge) : \text{WFF} \times \text{WFF} &\rightarrow \text{WFF}, & : (\varphi, \psi) &\mapsto \varphi \wedge \psi \\ (\vee) : \text{WFF} \times \text{WFF} &\rightarrow \text{WFF}, & : (\varphi, \psi) &\mapsto \varphi \vee \psi \\ (\Rightarrow) : \text{WFF} \times \text{WFF} &\rightarrow \text{WFF}, & : (\varphi, \psi) &\mapsto \varphi \Rightarrow \psi \end{aligned}$$

Remark 1.1 (Constructor vs Symbol View of Logical Connectives). Earlier we wrote $\varphi \Rightarrow \psi$ and then we denoted $\Rightarrow$ symbol as a function of type $\text{WFF} \times \text{WFF} \rightarrow \text{WFF}$, we use $(\Rightarrow)$ to distinguish between the two concepts.

A different but related question that arises is the ambiguity of $P_1 \Rightarrow P_2 \Rightarrow P_3$. According to the inductive definition there are two ways this formula could have been constructed which corresponds to the different formulas:

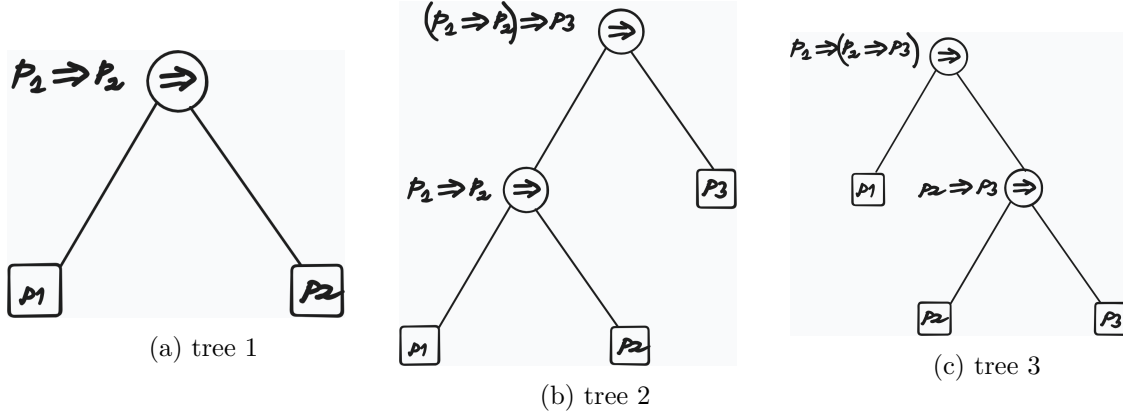
1. $(P_1 \Rightarrow P_2) \Rightarrow P_3$
2. $P_1 \Rightarrow (P_2 \Rightarrow P_3)$

To resolve this ambiguity we define the associativity rule:

Definition 1.3 (Associativity Rule for $\Rightarrow$).

$$P_1 \Rightarrow P_2 \Rightarrow P_3 := (P_1 \Rightarrow P_2) \Rightarrow P_3$$

Remark 1.2 (Wffs as Trees). Wffs can be viewed as trees. Trees encode the derivation structure and resolve the ambiguity



our collection of wffs can be further enriched with another binary constructor:

$$\begin{aligned}
 (\Leftrightarrow) : \text{WFF} \times \text{WFF} &\rightarrow \text{WFF} \\
 : (\varphi, \psi) &\mapsto \varphi \Leftrightarrow \psi
 \end{aligned}$$

Remark 1.3 (Meaning of $\Leftrightarrow$). At this stage there is no relation between

$$\varphi \Leftrightarrow \psi \quad \text{and} \quad (\varphi \Rightarrow \psi) \wedge (\psi \Rightarrow \varphi)$$

Let us give another ‘set-theoretic’ approach

1.4 Set Theoretic Approach

First we define the alphabets:

Definition 1.4 (Set Theoretic Approach to Defining Wffs). We start with the alphabets:

$$\begin{aligned}
 \mathcal{A} &:= \{P_i\}_{i \in \mathbb{N}} && \text{(sub-alphabet of atomic formulas)} \\
 \mathcal{C} &:= \{\Rightarrow, \wedge, \vee, \neg, \Leftrightarrow\} && \text{(sub-alphabet of logical connectives)} \\
 \mathcal{V} &:= \mathcal{A} \cup \mathcal{C} \cup \{(\,,\,)\} && \text{(alphabet of propositional logic, including punctuation marks)}
 \end{aligned}$$

Then we define the so-called **Kleene Closure** of $\mathcal{V}$:

Definition 1.5 (Kleene Closure of $\mathcal{V}$).

$$\mathcal{V}^* := \bigcup_{i=0}^{\infty} \mathcal{V}_i$$

$\mathcal{V}^*$ is the language containing all sorts of strings that can be constructed from the alphabet $\mathcal{V}$, also senseless ones.

To define the language propositional logic that contains only the formulas that we want and nothing else, we take the usual set-theoretic approach of taking the infinite union of all sets $\mathcal{W} \subseteq \mathcal{V}^*$ that satisfy the following properties:

- 1) $\mathcal{A} \subseteq \mathcal{W}$
- 2) if $\varphi, \psi \in \mathcal{W}$ then $\varphi \diamond \psi \in \mathcal{W}$, where $\diamond \in \{\Rightarrow, \wedge, \vee, \Leftrightarrow\}$
- 3) if $\varphi \in \mathcal{W}$ then $\neg\varphi \in \mathcal{W}$

Then the language of propositional logic is defined as:

Definition 1.6 (Language Propositional Logic as Infinite Intersection).

$$\mathcal{F}(\mathcal{A}) := \bigcap \{\mathcal{W} \subseteq \mathcal{V}^* \mid \mathcal{W} \text{ satisfies (1), (2), and (3)}\}$$

Theorem 1.1.

With this construction $\mathcal{F}(\mathcal{A})$ is the smallest set that satisfies the properties (1), (2) and (3), i.e.

- 1) $\mathcal{F}(\mathcal{A})$ satisfies the properties (1), (2) and (3)
- 2) For any set $\mathcal{W}$ that satisfies the properties (1), (2) and (3) we have $\mathcal{W} \subseteq \mathcal{F}(\mathcal{A})$

1.5 Summary

- WFF is defined inductively
- Constructors that generate WFFs
- Set theoretic construction of WFFs

2 Lecture 2 - Height and Subformulas

2.1 Overview

- We provide a fourth, iterative set construction for the language of propositional logic.
- We give two equivalent definitions for the height of a formula.
- We give two equivalent definitions of a function that gives the list of subformulas of a formula

2.2 Recursive Set Construction

We define $\mathcal{F}_0$ iteratively as follows:

$$\begin{aligned}\mathcal{F}_0 &:= \mathcal{A} \\ \mathcal{F}_1 &:= \mathcal{F}_0 \cup \{ \neg\varphi \mid \varphi \in \mathcal{F}_0 \} \cup \{ \varphi \diamond \psi \mid \varphi, \psi \in \mathcal{F}_0 \} \quad (\diamond \in \mathcal{C}) \\ \mathcal{F}_n &:= \mathcal{F}_{n-1} \cup \{ \neg\varphi \mid \varphi \in \mathcal{F}_{n-1} \setminus \mathcal{F}_{n-2} \} \cup \{ \varphi \diamond \psi \mid \varphi, \psi \in \mathcal{F}_{n-1} \} \quad (n \geq 2)\end{aligned}$$

With this construction it follows that:

$$\mathcal{F}_n \subseteq \mathcal{F}_{n+1}$$

Example 2.1. $\neg\neg P_4 \in \mathcal{F}_2 \subseteq \mathcal{F}_3 \subseteq \dots$

but $\neg\neg P_4 \notin \mathcal{F}_1$, because $\neg P_4 \notin \mathcal{F}_0$

Now we make the following definition:

Definition 2.1.

$$\mathcal{F} := \bigcup_{n \in \mathbb{N}} \mathcal{F}_n$$

and we claim:

Lemma 2.1.

$$\mathcal{F} = \mathcal{F}(\mathcal{A})$$

Where $\mathcal{F}(\mathcal{A})$ is defined in Definition 1.6

Proof. To prove $\mathcal{F} \subseteq \mathcal{F}(\mathcal{A})$, it suffices to show that

$$\mathcal{F}_n \subseteq \mathcal{F}(\mathcal{A}), \quad \forall n, \text{ by induction on } n$$

To conclude the proof we need to show that $\mathcal{F}_A \subseteq \mathcal{F}$ □

Definition 2.2 (Height of Formula - Set Approach). For all $\varphi \in \mathcal{F}$, there is a well defined natural number $\text{ht}(\varphi) \in \mathbb{N}$:

$$\text{ht}(\varphi) := \min\{n \in \mathbb{N} \mid \varphi \in \mathcal{F}_n\}$$

called the **height** of φ

We give another, a more computational definition for the height.

Definition 2.3 (Height of a Formula - Computational Approach). We define $\text{ht}(\cdot)$ recursively on the structure of a formula ϕ as follows:

$$\begin{aligned} \text{ht}(P_i) &= 0 & (P_i \in \mathcal{A}) \\ \text{ht}(\neg\varphi) &= 1 + \text{ht}(\varphi) \\ \text{ht}(\varphi \diamond \psi) &= 1 + \max(\text{ht}(\varphi), \text{ht}(\psi)) \quad (\diamond \in \mathcal{C}) \end{aligned}$$

Remark 2.1 (Pattern Matching). Such inductive (recursive) definitions are also called **pattern matching**

Example 2.2 (Derivation of Height).

$$\begin{aligned} \text{ht}(\neg\neg(P_1 \Rightarrow P_2)) &= 1 + \text{ht}(\neg(P_1 \Rightarrow P_2)) \\ &= 1 + 1 + \text{ht}(P_1 \Rightarrow P_2) \\ &= 1 + 1 + \max(\text{ht}(P_1), \text{ht}(P_2)) \\ &= 1 + 1 + \max(1, 1) \\ &= 1 + 1 + 1 \\ &= 3 \end{aligned}$$

Observation:

We observe that the height of a formula is the height of its syntax tree

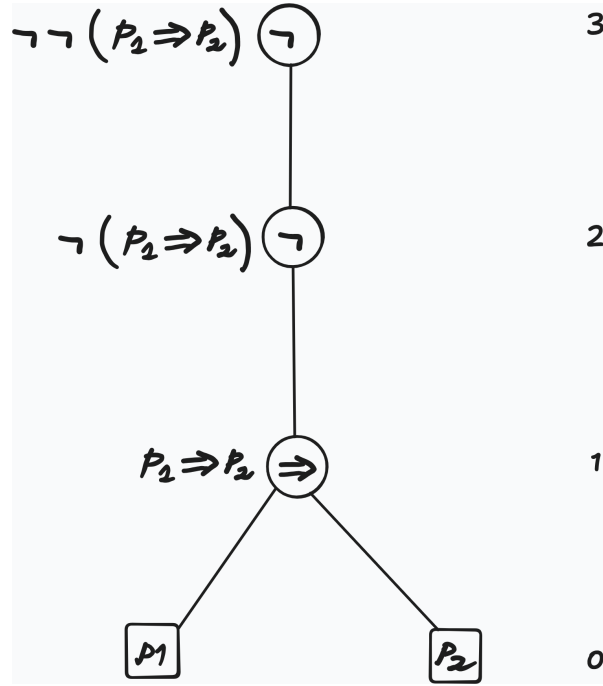


Figure 2.1: height of the syntax tree

We want to define a function

$$\text{sf} : \mathcal{F}(\mathcal{A}) \rightarrow \mathcal{P}(\mathcal{F}(\mathcal{A}))$$

mapping a formula to its set of subformulas. For this we are going to define functions $\text{sf}_i : \mathcal{F}_i \rightarrow \mathcal{P}(\mathcal{F})$ as follows:

Definition 2.4 (Set Construction of sf).

$$\begin{aligned} \text{sf}_0(P_i) &:= \{P_i\} \\ \text{sf}_n(\neg\varphi) &:= \{\neg\varphi\} \cup \text{sf}_{n-1}(\varphi) \\ \text{sf}_n(\varphi \diamond \psi) &:= \{\varphi \diamond \psi\} \cup \text{sf}_{n-1}(\varphi) \cup \text{sf}_{n-1}(\psi) \end{aligned}$$

Then sf can be defined as

$$\begin{aligned} \text{sf} : \mathcal{F} &\rightarrow \mathcal{P}(\mathcal{F}) \\ &: \varphi \mapsto \text{sf}_{\text{ht}(\varphi)}(\varphi) \end{aligned}$$

We give another, more computational approach, by defining sf recursively on the structure of wffs as a function $\text{sf} : \text{WFF} \rightarrow [\text{WFF}]$

Definition 2.5 (List Construction of sf).

$$\begin{aligned}\text{sf}(P_i) &:= [P_i] \\ \text{sf}(\neg\varphi) &:= [\neg\varphi] ++ \text{sf}(\varphi) \\ \text{sf}(\varphi \diamond \psi) &:= [\varphi \diamond \psi] ++ \text{sf}(\varphi) ++ \text{sf}(\psi)\end{aligned}$$

Example 2.3 (Derivation of the List of Subformulas with sf).

$$\begin{aligned}\text{sf}(P_1 \vee \neg P_2) &= [P_1 \vee \neg P_2] ++ \text{sf}(P_1) ++ \text{sf}(\neg P_2) \\ &= [P_1 \vee \neg P_2] ++ [P_1] ++ [\neg P_2] ++ \text{sf}(P_2) \\ &= [P_1 \vee \neg P_2] ++ [P_1] ++ [\neg P_2] ++ [P_2] \\ &= [P_1 \vee \neg P_2, P_1, \neg P_2, P_2]\end{aligned}$$

2.3 Summary

- iterative set construction $\mathcal{F}_i$ - the language of propositional logic as infinite union of such
- two definitions for ht - height of a formula
- two definitions for sf - list or set of subformulas of a formula

References

3 Resources

Books & other resources

3.1 SML

- Elements of Functional Programming. Reade
- Introduction to Programming Using SML. Hansel, Rischel
- Programming in SML. Harper
- Programmierung: Eine Einfuehrung in Standard ML. Gert Smolka

3.2 Haskell

- Functional Programming with Gofer. Fokker
- Introduction to Computation, Haskell, Logic, and Automata. Wadler
- Introduction to Functional Programming using Haskell. Bird
- Introduction to Functional Programming Systems Using Haskell
- Thinking Functionally with Haskell. Bird
- Programming in Haskell. Graham Hutton
- Haskell - The Craft of Functional Programming. Thompson

3.3 OCAML

- OCaml from the very beginning. Whittington
- More OCaml. Whittington
- OCaml - Functional Programming for the Masses. Madhavapeddy

3.4 Rocq / Coq

- Mathematical Components. Assia Mahboubi, Enrico Tassi.
- Software Foundations. Benjamin C. Pierce
- Programs and Proofs - Mechanizing Mathematics with Dependent Types. Ilya Sergey
- Certified Programming with Dependent Types. Adam Chipala
- Interactive Theorem Proving and Program Development - Coq'Art: The Calculus of Inductive Constructions. Bertot, Casteran